TECHNICAL SPECIFICATION & ACADEMIC PROJECT REPORT

Software Requirements Specification (SRS) for Sanitary & Ceramic Factory Workforce & Payroll Management System

A High-Integrity Cross-Platform Flutter Client & Django REST Enterprise Architecture for Industrial Manufacturing Units

EXECUTIVE SUMMARY & SYSTEM PURPOSE

This engineering specification establishes the complete functional boundaries, relational database schema, RESTful API contracts, atomic transaction constraints, mathematical payroll calculation models, and user interaction standards for the Sanitary Factory Workforce and Payroll ERP.

Attribute	Specification / Academic Submission Details
Project Title	Sanitary & Ceramic Factory Workforce & Payroll ERP (SanitaryApp)
Document Standard	IEEE Std 830-1998 Recommended Practice for SRS
Document Version	Version 1.0.0 (Engineering Baseline)
Release Date	October 2026 (Academic Session 2026–2027)
Candidate / Student Name	[Student Name / Candidate Name]
Roll Number / Enrolment ID	[Roll Number / Student ID Placeholder]
Degree / Course	[B.Tech / B.E. / M.C.A. in Computer Science & Engineering]
Subject / Paper	[Software Engineering / Capstone Industrial Project Implementation]
Academic Institution	[University Institute of Technology / Faculty of Engineering]

Application Stack	Flutter 3.x (Dart), Django 5.x / 6.x (Python 3.10+), PostgreSQL / SQLite

TABLE OF CONTENTS

Part 1 — Project Introduction
1.1 Project Introduction
1.2 Need for the Project
1.3 Existing Problems and Proposed Solution
1.4 Project Objectives
1.5 Project Scope (In-Scope & Future Enhancements)
Part 2 — Software Requirements
2.1 Functional Requirements (FR-AUTH, FR-DEPT, FR-SHFT, FR-STF, FR-ATT, FR-ADV, FR-PAY, FR-DASH)
2.2 Non-Functional Requirements (Performance, Reliability, Security, Usability, Maintainability)
2.3 User Roles and Permissions (Role-Based Access Control Matrix)
2.4 Input, Processing, and Output Requirements
Part 3 — Hardware and Software Requirements
3.1 Hardware and Software Technology Stack Table
Part 4 — System Architecture and Workflows
4.1 System Architecture Diagram & Layered Model
4.2 Overall Application Flowchart
4.3 Module-Specific Workflows (Daily Roster, Salary Advances, Payroll Engine)
Part 5 — Database Design
5.1 Database Overview & RDBMS Selection
5.2 Relational Database Tables DDL & Constraints
5.3 Entity Relationship Diagram (ERD)
5.4 Data Relationships and Integrity Rules
Part 6 — Interface and Screen Documentation
6.1 Screen Profiles & Detailed Interaction Specifications
Part 7 — Testing Requirements
7.1 Comprehensive Verification & Test Cases Matrix
Part 8 — Security and Limitations
8.1 Security Architecture, Cryptography & Token Lifecycle
8.2 Financial Integrity & Concurrency Controls
8.3 Current Technical Limitations & Implementation Gaps
Part 9 — Conclusion and Future Scope
9.1 Project Conclusion
9.2 Future Scope & Planned Industrial Integrations

PART 1 — PROJECT INTRODUCTION

1.1 Project Introduction

The Sanitary & Ceramic Factory Workforce & Payroll Management System (SanitaryApp) is an industrial-grade enterprise application designed specifically to address the complex operational dynamics of ceramic, sanitaryware, and tile manufacturing facilities. In continuous manufacturing plants, production runs 24 hours a day across distinct rotating shifts (such as Morning, Evening, and Night shifts) across specialized departments including Moulding, Casting, Kiln, Glazing, Quality Inspection, and Packaging.

Manufacturing workers in these units operate under distinct wage structures, frequent overtime demands, and mid-month salary advance dependencies. The SanitaryApp platform provides a unified digital operational hub comprising a high-performance cross-platform Flutter client deployed at shop-floor supervisor desks and manager terminals, paired with a robust Django REST Framework backend operating over an ACID-compliant relational database. The platform completely eliminates paper registers, prevents payroll disputes, enforces strict audit trails, and delivers real-time visibility into workforce deployment and factory labor expenditures.

1.2 Need for the Project

Traditional sanitary and ceramic manufacturing plants in industrial corridors rely heavily on physical paper muster rolls, manual punch registers, or disconnected Excel spreadsheets. In an environment with hundreds of plant workers, high ambient dust, kiln heat, and continuous shift changes, these manual methods exhibit severe structural limitations:

- **Muster Roll Degradation:** Daily attendance tracking on physical log sheets is prone to damage, unauthorized alterations, illegible handwriting, and inaccurate attendance reconciliation.
- **Overtime Calculation Leakage:** Factory workers frequently log 2 to 6 hours of overtime during kiln firing or peak dispatch schedules. Manually aggregating overtime hours at month-end across hundreds of daily sheets leads to arithmetic errors and labor dissatisfaction.
- **Uncollected Salary Advances:** Workers frequently take mid-month cash advances for emergencies. When cash disbursements are recorded in informal physical diaries, they are often forgotten during month-end payroll, causing direct financial leakage to the enterprise.
- **Laborious Month-End Payroll Run:** Calculating individual base pay based on dynamic attendance weights (Full-Day = 1.0, Half-Day = 0.5), overtime hourly rates, and mid-month deductions takes days of manual accounting, resulting in delayed paydays and operational downtime.
- **Lack of Real-Time Operational Visibility:** Plant managers and factory owners have zero real-time visibility into shop-floor attendance percentages, daily absenteeism, or month-to-date labor expenditure.

1.3 Existing Problems and Proposed Solution

To establish a clear engineering justification, Table 1.1 contrasts the manual/spreadsheet workflow against the automated solution implemented in the SanitaryApp system.

Operational Dimension	Legacy Manual / Spreadsheet Approach	SanitaryApp Digital ERP Solution
Daily Attendance Marking	Paper muster roll signed or ticked by supervisors; high risk of proxy marking and lost records.	Mobile/Tablet Daily Roster with single-tap status marking (Present, Half-Day, Absent, Leave) and atomic bulk commit.

Overtime Recording	Handwritten chits or separate overtime notebooks, leading to discrepancies and disputes.	Direct overtime hours entry (0.00 to 12.00 hrs) integrated into the daily roster with hourly rate multiplier.
Salary Advance Tracking	Physical cash vouchers stored in accounting drawers; high incidence of unrecovered loans.	Formal Advance Salary Ledger tracking disbursed amounts, dates, and automatic status transition (APPROVED to DEDUCTED).
Payroll Formulation	Manual multi-column Excel spreadsheets vulnerable to formula tampering, cell errors, and version drift.	Automated 30-day payroll computation engine executing validated formula: Net = BasePayable + OT - Advances.
Auditability & History	No immutable records; changes to past attendance or pay slips leave no trace.	PostgreSQL / SQLite relational database with created_at, updated_at timestamps, immutable draft-to-finalized lifecycle.
Executive Insights	Manual compilation of reports taking days; historical metrics unavailable on demand.	Real-time Executive Dashboard displaying live attendance counts, OT hours, monthly payroll projections, and cash advances.

1.4 Project Objectives

The principal engineering objectives accomplished by this project are:

- **1. Multi-Platform Shop Floor Deployment:** Develop an intuitive, touch-friendly Flutter client application suitable for mobile phones, rugged tablets, and desktop workstations.
- **2. Sub-Second Roster Management:** Implement atomic bulk attendance marking allowing supervisors to log up to 500 active workers in under 1 second with half-day and overtime tracking.
- **3. Financial Advance Reconciliation:** Build an immutable advance salary ledger that links disbursements to workers and deducts approved balances during payroll computation.
- **4. Automated Payroll Calculation Engine:** Implement a mathematically verified 30-day industrial wage computation engine that calculates base pay, overtime pay, and net pay.
- **5. Role-Based Security Governance:** Enforce strict Role-Based Access Control (RBAC) separating Supervisor, Manager, and Admin privileges using JSON Web Tokens (JWT).
- **6. Executive Operational Visibility:** Provide aggregated Key Performance Indicators (KPIs) through a modern, responsive dashboard interface.

1.5 Project Scope

The project scope encompasses all functional and technical boundaries established in the current codebase:

In-Scope Functionality (Current Implementation): The system fully implements: (1) JWT authentication and user management with three distinct roles; (2) Department and Shift master configurations; (3) Complete Staff Master Directory with employee codes, designations, monthly salaries, and overtime rates; (4) Daily Attendance Roster with bulk atomic upsert and attendance history filters; (5) Mid-month Advance Salary issuance and deduction lifecycle; (6) Formula-driven monthly payroll computation, draft review, individual/bulk finalization, and detailed payslip generation; (7) Executive KPI Dashboard with live daily statistics and monthly payroll cost projections.

Out-of-Scope / Future Enhancements: Planned future additions outside the initial baseline: (1) Direct TCP/IP and USB hardware driver integration for biometric fingerprint and facial recognition turnstiles; (2) Direct integration with banking Open APIs (e.g. RazorpayX / Cashfree) for automated electronic salary payouts; (3) Native employee mobile self-service application; (4) Multi-factory enterprise clustering and multi-currency payroll support.

PART 2 — SOFTWARE REQUIREMENTS

2.1 Functional Requirements

The functional requirements specify the complete set of operational capabilities provided by the application. Each module is described with its formal identifier, purpose, user actions, inputs, processing logic, outputs, and validation rules.

Module 1: Authentication & Role-Based Access Control (FR-AUTH)

Purpose: Manages system security, identity verification, and role-based operational permissions across the factory.

- **FR-AUTH-01 (JWT Login):** Users submit username and password credentials; system issues signed JWT access and refresh token pair upon verification.
- **FR-AUTH-02 (Token Rotation):** System automatically refreshes expired access tokens (24-hour lifetime) using long-lived refresh tokens (7-day lifetime) without user interruption.
- **FR-AUTH-03 (Admin Privileges):** Access to administrative features (user provisioning, payroll generation, payroll finalization) is strictly restricted to ADMIN role.
- **FR-AUTH-04 (Role Masking):** Supervisors are restricted to attendance marking and staff directory viewing; financial fields (salaries, advances) are masked.

Field / Stage	Specification
Inputs	Username (string, max 150 chars), Password (string, min 6 chars)
Processing	Django authenticate() -> SimpleJWT token generation -> claims packaging {user_id, role}
Outputs	JSON response containing access_token, refresh_token, and user profile metadata
Validations	Mandatory non-empty fields; invalid credentials return HTTP 401 Unauthorized
Expected Result	Authorized user transitions to MainNavigationShell; tokens persisted securely in SharedPreferences

Module 2: Department & Shift Management (FR-DEPT / FR-SHFT)

Purpose: Defines factory organizational structure and work shift schedules for grouping workers and roster organization.

- **FR-DEPT-01 (Department Setup):** Admins and Managers can create, update, and deactivate factory production departments (e.g. Moulding, Kiln, Glazing).
- **FR-DEPT-02 (Department Integrity):** Departments maintain unique names; active status toggling prevents assigning workers to decommissioned divisions.
- **FR-SHFT-01 (Shift Configuration):** Configuration of plant shift schedules with unique shift names, standard start times, and end times (e.g. Morning 08:00–16:00).
- **FR-SHFT-02 (Shift Assignment):** Shift foreign keys in staff records dictate daily roster scheduling and supervisor filtering.

Field / Stage	Specification
Inputs	Department: Name (unique, max 100), Description; Shift: Name (unique, max 50), start_time, end_time
Processing	Validation of unique constraints -> Persistence via Django ORM -> ordering by start_time / name
Outputs	Department and Shift JSON entities returned to Flutter dropdowns and configuration screens
Validations	Duplicate department or shift names trigger HTTP 400 with descriptive error message
Expected Result	Immediate reflection across staff onboarding and daily roster department filters

Module 3: Staff Master Directory & Profile Lifecycle (FR-STF)

Purpose: Central worker registry storing personal identity, departmental placement, shift assignment, and wage contracts.

- **FR-STF-01 (Worker Registration):** Managers can onboard new factory workers by entering employee code, full name, phone, joining date, department, shift, monthly salary, and overtime rate.
- **FR-STF-02 (Unique Identity):** Employee codes must be globally unique across the factory (e.g. STF001, STF002) and indexed for instant lookups.
- **FR-STF-03 (Wage Rate Validation):** Base monthly salary and hourly overtime rate must be strictly non-negative; validated at model, serializer, and UI layers.
- **FR-STF-04 (Status Lifecycle):** Workers can be transitioned between ACTIVE and INACTIVE status; inactive workers are excluded from future rosters.
- **FR-STF-05 (Search & Filter):** Staff directory provides search by name/code and filtering by department, shift, and status.

Field / Stage	Specification
Inputs	employee_code, full_name, phone, joining_date, department_id, shift_id, monthly_salary, overtime_rate, status
Processing	Field validation -> CheckConstraints evaluation -> ORM insert/update -> Return complete record
Outputs	Staff entity JSON with computed daily_salary property (monthly_salary / 30)
Validations	Unique employee_code; monthly_salary >= 0; overtime_rate >= 0; valid FKs
Expected Result	Staff profile saved; worker immediately appears in department rosters and master lists

Module 4: Daily Roster & Bulk Attendance Operations (FR-ATT)

Purpose: Facilitates rapid shop-floor attendance capture, status designation, and overtime recording for hundreds of workers.

- **FR-ATT-01 (Roster Retrieval):** Supervisors load daily roster for any target date and optional department filter; displays all active workers.
- **FR-ATT-02 (Status Weights):** Status designation per worker supports four standard industrial states: PRESENT (weight 1.0), HALF_DAY (weight 0.5), ABSENT (weight 0.0), LEAVE (weight 0.0).
- **FR-ATT-03 (Overtime Capture):** Overtime recording allows supervisors to input incremental overtime hours between 0.00 and 12.00 hours per day.
- **FR-ATT-04 (Atomic Bulk Commit):** Bulk submission executes via atomic database transaction (@transaction.atomic), preventing partial saves.
- **FR-ATT-05 (Upsert Guarantee):** Composite unique constraint (staff_id, date) enforces upsert behavior (update existing, insert if new).
- **FR-ATT-06 (Daily Attendance Summary):** Summary endpoint calculates present count, half-day count, absent count, leave count, and total overtime hours.

Field / Stage	Specification
Inputs	Target Date (YYYY-MM-DD), Array of items {staff_id, status, overtime_hours, remarks}
Processing	transaction.atomic() loop -> Attendance.objects.update_or_create() -> Aggregation update
Outputs	JSON confirmation: {success: true, saved_count: N, date: YYYY-MM-DD}
Validations	0.00 <= overtime_hours <= 12.00; status in ['PRESENT','HALF_DAY','ABSENT','LEAVE']
Expected Result	All records committed; instant update of attendance history and live dashboard metrics

Module 5: Salary Advances & Loan Ledger (FR-ADV)

Purpose: Tracks mid-month emergency cash advances disbursed to factory workers and automates their deduction at month-end.

- **FR-ADV-01 (Advance Recording):** Managers can record cash advance disbursements by selecting staff member, amount, date, and reason.
- **FR-ADV-02 (Positive Amount Rule):** Advance amount must be strictly positive (amount > 0.00); enforced by check constraint.
- **FR-ADV-03 (Approved State):** Disbursements initialize in APPROVED status and represent outstanding obligations against future payroll.
- **FR-ADV-04 (Automated Payroll Deduct):** During monthly payroll computation, all APPROVED advances for that staff member and month are automatically aggregated.
- **FR-ADV-05 (Lifecycle Locking):** When payroll is finalized, associated advance records transition to DEDUCTED status, preventing double deduction.

Field / Stage	Specification
Inputs	staff_id (FK), amount (Decimal > 0), date (Date), reason (string, max 255)
Processing	Model check constraint validation -> AdvanceSalary.objects.create(status='APPROVED')
Outputs	Advance record JSON with staff details, formatted currency amount, and lifecycle status
Validations	Amount > 0; valid staff reference; date cannot be blank
Expected Result	Advance logged in ledger; reflected in worker's profile and active month's payroll preview

Module 6: Automated Monthly Payroll Computation Engine (FR-PAY)

Purpose: Mathematical calculation and locking of monthly worker wages based on attendance, overtime hours, and loan deductions.

- **FR-PAY-01 (Batch Calculation):** Admins and Managers can trigger batch payroll calculation for any selected month (1–12) and year.
- **FR-PAY-02 (30-Day Formulation):** Calculations adhere strictly to standard 30-day industrial wage formulation: DailyWage = BaseSalary / 30.00.
- **FR-PAY-03 (Paid Days Summation):** Paid days derived from attendance records: PaidDays = PresentCount * 1.0 + HalfDayCount * 0.5.
- **FR-PAY-04 (Overtime Pay):** Overtime pay computed: OvertimeAmount = OvertimeHours * OvertimeRate.
- **FR-PAY-05 (Net Salary Formula):** Net salary computed: NetSalary = max(0.00, BasePayable + OvertimeAmount - AdvanceDeduction - OtherDeductions).
- **FR-PAY-06 (Draft Review Stage):** Records generated initially in DRAFT status, allowing HR managers to review and adjust deductions.
- **FR-PAY-07 (Payroll Finalization):** Admin executes finalize / finalize_all to transition status to FINALIZED, lock values, and mark advances as DEDUCTED.

Field / Stage	Specification
Inputs	month (1–12), year (integer e.g. 2026), optional department_id
Processing	PayrollCalculator iterates active staff -> aggregates attendance & advances -> creates/updates Payroll
Outputs	Payroll list JSON with complete itemization: base, paid_days, OT, advances, gross, and net salary
Validations	Month in 1..12; Net salary >= 0; Finalized records cannot be modified or re-generated
Expected Result	Draft payroll records generated; summary totals computed; printable payslips ready

Module 7: Executive Dashboard & Factory Analytics (FR-DASH)

Purpose: Delivers high-level operational visibility, attendance percentages, and financial summaries to management.

- **FR-DASH-01 (Workforce Metrics):** Live KPI cards show: Active Staff Headcount, Inactive Staff Headcount, Department and Shift totals.
- **FR-DASH-02 (Live Attendance Tracking):** Today's live attendance card shows real-time Present, Half-Day, Absent, Leave, Unmarked counts, and OT hours.
- **FR-DASH-03 (Financial Exposure):** Monthly financial overview shows total projected net payroll, draft vs. finalized count, and advance disbursements.

Field / Stage	Specification
Inputs	HTTP GET request to /api/dashboard/kpi/ with Bearer authorization header
Processing	Django aggregates Staff, Attendance(date=today), Payroll(month=current), and AdvanceSalary
Outputs	JSON object containing factory_overview, today_attendance, and current_month_payroll
Validations	Requires valid authenticated session; auto-refreshes on dashboard pull-to-refresh
Expected Result	Instant visualization of operational health on mobile, tablet, and desktop screens

2.2 Non-Functional Requirements

Non-functional requirements specify qualitative engineering criteria, performance benchmarks, and architectural constraints.

NFR Identifier	Dimension	Requirement Specification & Verification Metric
NFR-PERF-01	Roster Latency	Loading daily attendance roster for up to 500 active staff members must complete with response time under 350 ms.
NFR-PERF-02	Bulk Mark Latency	Submitting bulk attendance for 500 workers via POST /api/attendance/bulk_mark/ must commit in under 1000 ms.
NFR-PERF-03	Payroll Run Speed	Executing batch monthly payroll generation for an entire factory workforce must complete within 3.0 seconds.
NFR-REL-01	Transactional ACID	All multi-row updates (attendance bulk marking, payroll finalization) must execute inside database transactions.
NFR-REL-02	Referential Safety	All key foreign keys must enforce ON DELETE PROTECT, preventing accidental deletion of historical audit data.

NFR-SEC-01	Network Encryption	All communication between Flutter clients and backend API must be encrypted using TLS 1.3 / HTTPS.
NFR-SEC-02	Password Security	User passwords must be salted and hashed using PBKDF2 with SHA-256 (Django default security standard).
NFR-SEC-03	Token Expiry	JWT Access tokens expire after 24 hours; refresh tokens expire after 7 days with automatic silent rotation.
NFR-USE-01	Industrial Touch UI	Touch targets on Flutter client must be minimum 48x48 dp to facilitate rapid tapping on shop-floor tablets.
NFR-MNT-01	Modularity	Backend codebase structured into decoupled domain apps (accounts, staff, attendance, advances, payroll, dashboard).
NFR-CMP-01	Cross-Platform	Frontend single codebase runs natively across Android 10+, Windows 10/11 Desktop, iOS, and Web browsers.
NFR-AVL-01	High Availability	System architecture supports 99.9% uptime during operational plant hours (24x7 rotating industrial shifts).

2.3 User Roles and Permissions (RBAC Matrix)

The system enforces Role-Based Access Control (RBAC) across three distinct user roles: Admin (Factory Owner / General Manager), Manager (HR & Operations Lead), and Supervisor (Shop-Floor Line In-Charge). Table 2.3 defines the authoritative permissions matrix.

Operational Capability	Supervisor	Manager	Admin / Owner
View Executive KPI Dashboard	Allowed	Allowed	Allowed
View Staff Directory	Allowed (Masked Salaries)	Allowed (Full Wages)	Allowed (Full Wages)
Onboard / Edit Staff Members	Denied	Allowed	Allowed
Activate / Deactivate Staff	Denied	Allowed	Allowed
Manage Departments & Shifts	View Only	Full Management	Full Management
Load Daily Attendance Roster	Allowed	Allowed	Allowed
Submit Bulk Attendance & OT	Allowed	Allowed	Allowed
View Historical Attendance	Allowed	Allowed	Allowed

Disburse Salary Advances	Denied	Allowed	Allowed
Generate Draft Monthly Payroll	Denied	Allowed	Allowed
Finalize Payroll & Lock Payslips	Denied	Denied	Allowed
User Account Provisioning & Roles	Denied	Denied	Allowed

2.4 Input, Processing, and Output Requirements

System operations transform raw field inputs into verified business ledgers according to formal mathematical and algorithmic rules.

Mathematical Payroll Formulation: Daily Salary = Base Monthly Salary / 30.00
Paid Days = (Present Count * 1.0) + (Half Day Count * 0.5)
Base Payable = Daily Salary * Paid Days
Attendance Deduction = (30.00 - Paid Days) * Daily Salary
Overtime Amount = Total Overtime Hours * Hourly Overtime Rate
Gross Salary = Base Monthly Salary + Overtime Amount
Advance Deduction = Sum(Approved Advances for Target Month)
Net Payable Salary = max(0.00, Base Payable + Overtime Amount - Advance Deduction - Other Deductions)

PART 3 — HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.1 provides the complete hardware, software, and deployment environment specifications for both client devices and backend application servers.

Category	Requirement
Application type	Cross-Platform Industrial Workforce & Payroll Enterprise System (Client-Server)
Frontend	Flutter 3.x Framework (Material 3 Design, Google Fonts Outfit / Inter)
Programming language	Dart 3.x (Frontend Client), Python 3.10+ / 3.13 (Backend API)
Backend	Django 5.x / 6.x with Django REST Framework (DRF) 3.18.x and SimpleJWT 5.5.x
Database	PostgreSQL 15+ (Production Relational DBMS) / SQLite 3 (Local Staging & Testing)
Development environment	Visual Studio Code, Android Studio, Flutter SDK, Python Virtual Environment (venv)
Operating system	Client: Windows 10/11 Desktop, Android 10+ Tablet/Phone; Server: Ubuntu Linux 22.04 LTS / Windows Server
Server or hosting	Gunicorn / Uvicorn WSGI/ASGI Server behind Nginx Reverse Proxy with systemd service supervisor
APIs and integrations	RESTful JSON APIs over HTTP/HTTPS, SimpleJWT Bearer Authentication, Dio HTTP Client with Interceptors
Internet connectivity	Local Area Network (LAN) / Factory Wi-Fi or Cellular Internet (3G/4G/5G) for API communication

PART 4 — SYSTEM ARCHITECTURE AND WORKFLOWS

4.1 System Architecture Diagram

The SanitaryApp platform implements a strictly decoupled 4-tier enterprise client-server architecture: (1) Presentation Layer running on cross-platform Flutter; (2) Communication & Gateway Layer over HTTPS REST; (3) Application Logic Layer powered by modular Django domain apps; and (4) Relational Persistence Layer backed by PostgreSQL. Figure 4.1 illustrates the structural components and their communication vectors.

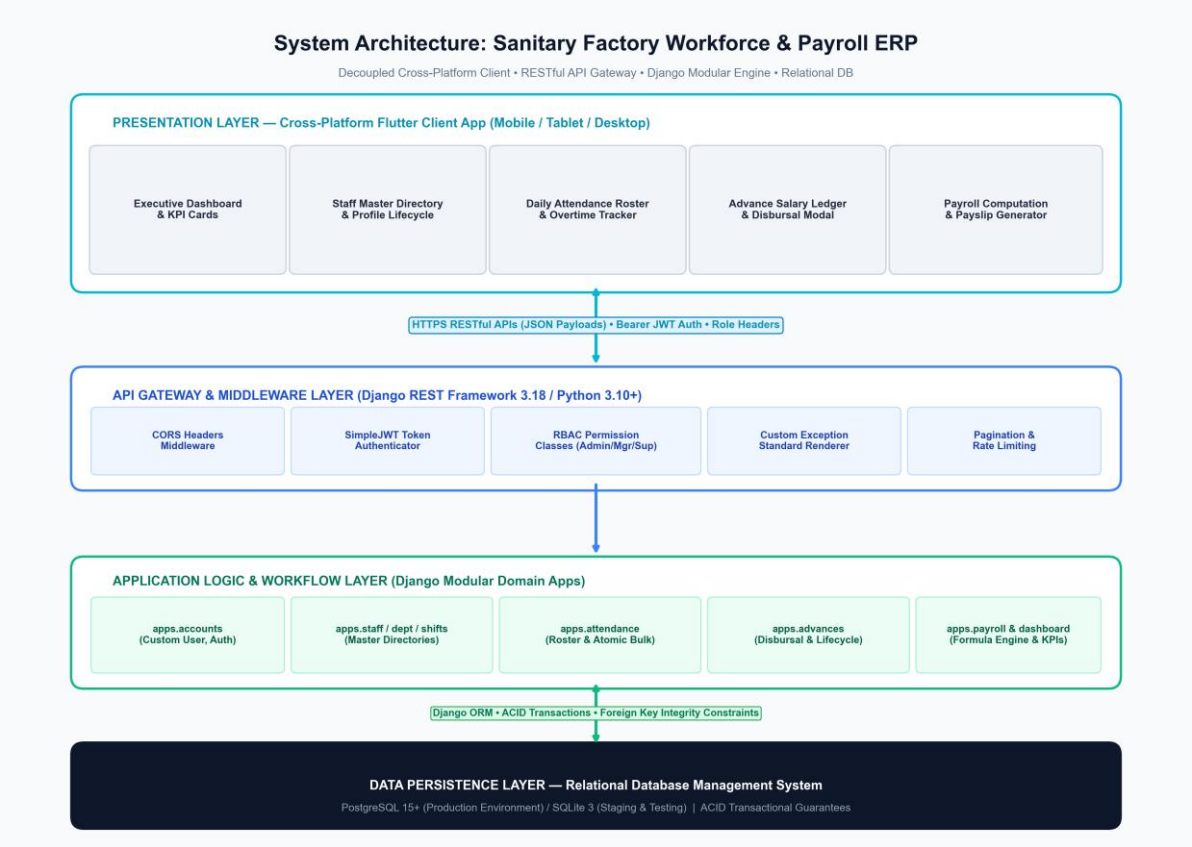


Figure 4.1: Decoupled 4-Tier System Architecture of the Sanitary Factory ERP

The Presentation Layer uses Flutter with Riverpod state management, maintaining a reactive unidirectional data flow between UI widgets, state notifiers, and repository classes. Network communication is handled by a singleton Dio client configured with authentication interceptors that automatically attach Bearer JWT tokens to outgoing requests and catch 401 unauthorized events to trigger token refresh workflows. The backend application server employs Django REST Framework viewsets protected by custom permission classes (IsAdminRole, IsManagerOrAdmin, IsSupervisorOrAbove) and standardized JSON envelope renderers.

4.2 Overall Application Flowchart

Figure 4.2 documents the end-to-end user navigation lifecycle, starting from operator authentication, through daily operational workflows, to monthly payroll locking and session termination.

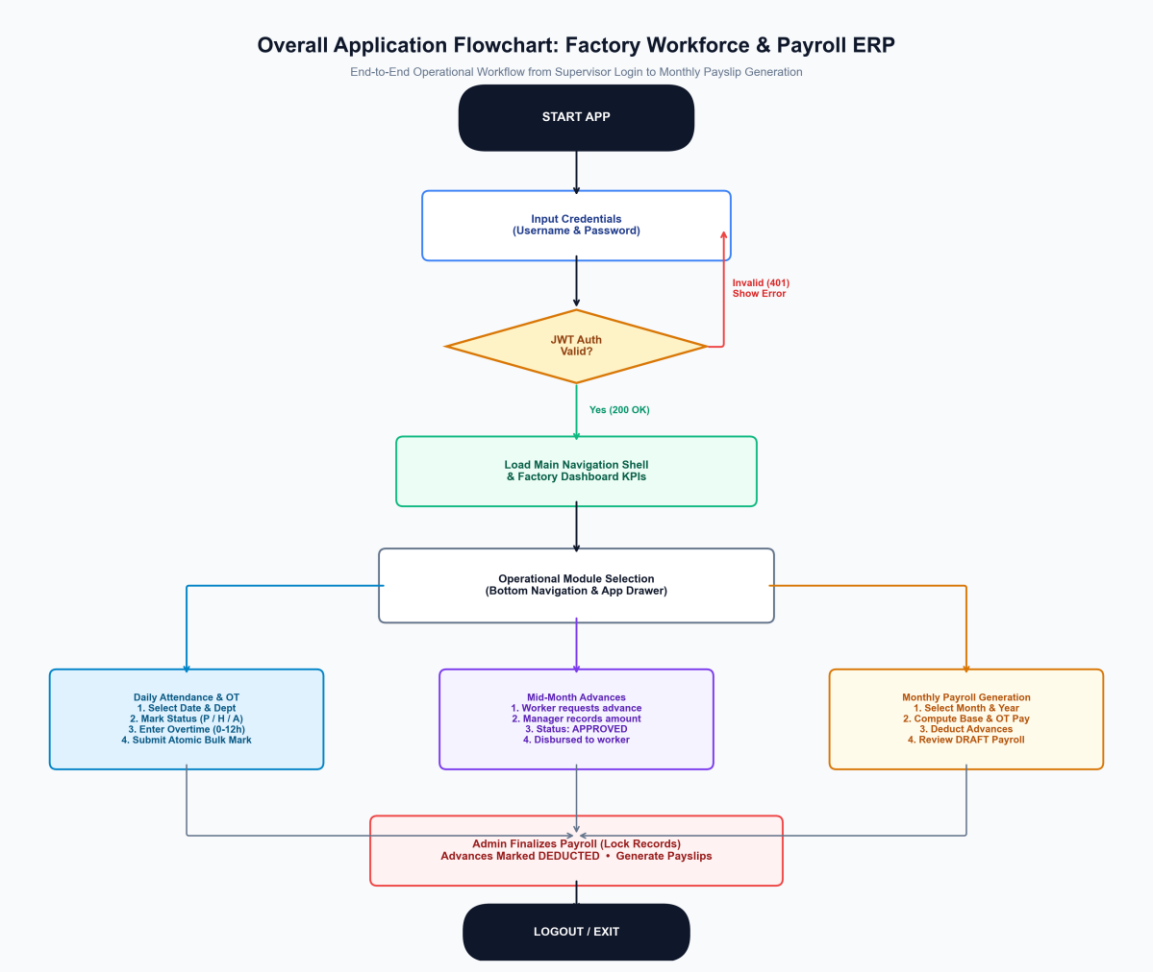


Figure 4.2: Overall Application Lifecycle and Module Navigation Flowchart

4.3 Module-Specific Workflows

To provide thorough coverage of critical factory operations, the following subsections illustrate the precise state transitions and data handling rules for the three core operational modules.

4.3.1 Daily Attendance & Overtime Processing Workflow

Daily attendance is captured using a bulk roster interface. Figure 4.3 traces the sequence from supervisor date selection to database transaction commitment.

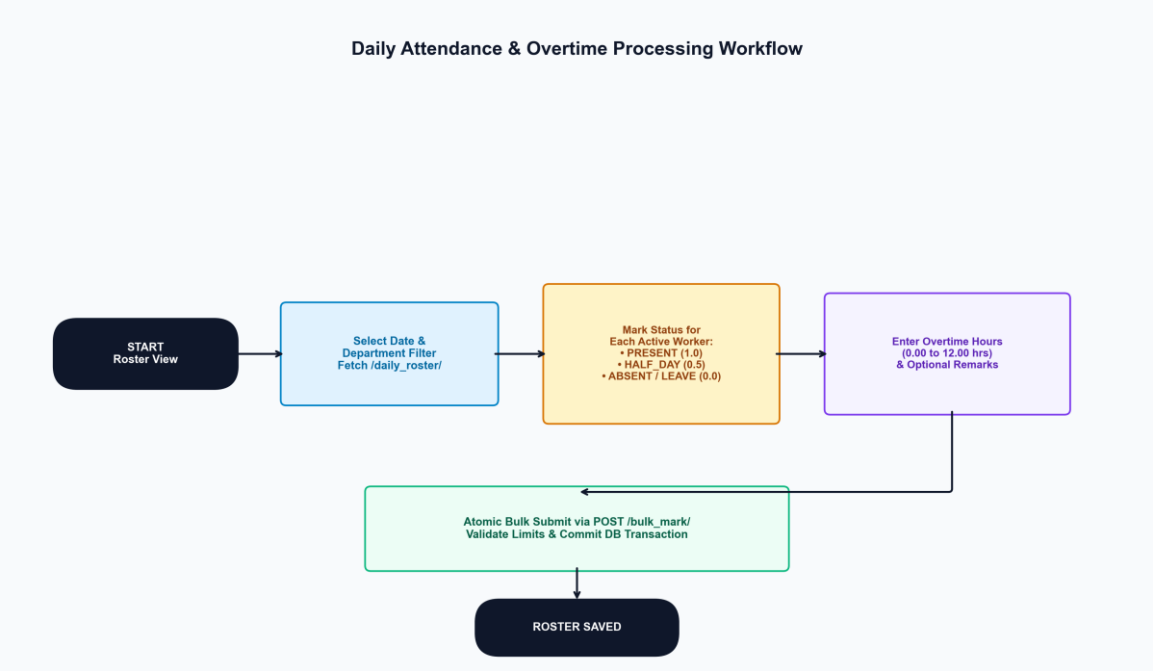


Figure 4.3: Daily Attendance Roster and Overtime Processing Flowchart

4.3.2 Mid-Month Salary Advance Lifecycle State Machine

Salary advances transition through strict lifecycle states: APPROVED upon disbursal, DEDUCTED upon month-end payroll finalization, or CANCELLED if voided prior to calculation. Figure 4.4 details this state machine.

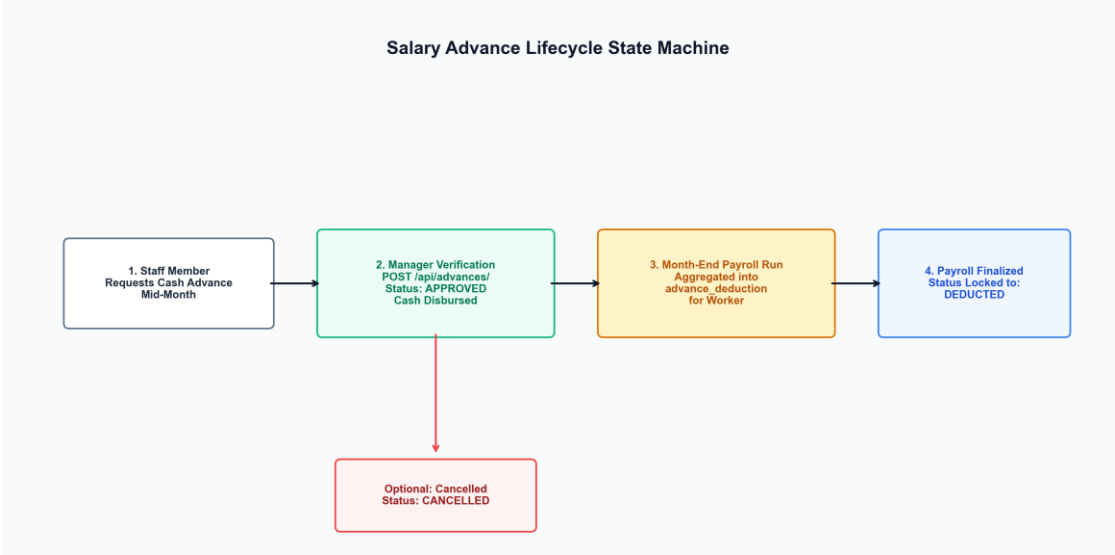


Figure 4.4: Salary Advance Lifecycle State Transition Diagram

4.3.3 Monthly Payroll Generation & Finalization Workflow

Figure 4.5 details the batch calculation sequence executed by the Django PayrollCalculator engine, including attendance aggregation, overtime multiplication, advance deduction, draft review, and administrative finalization.

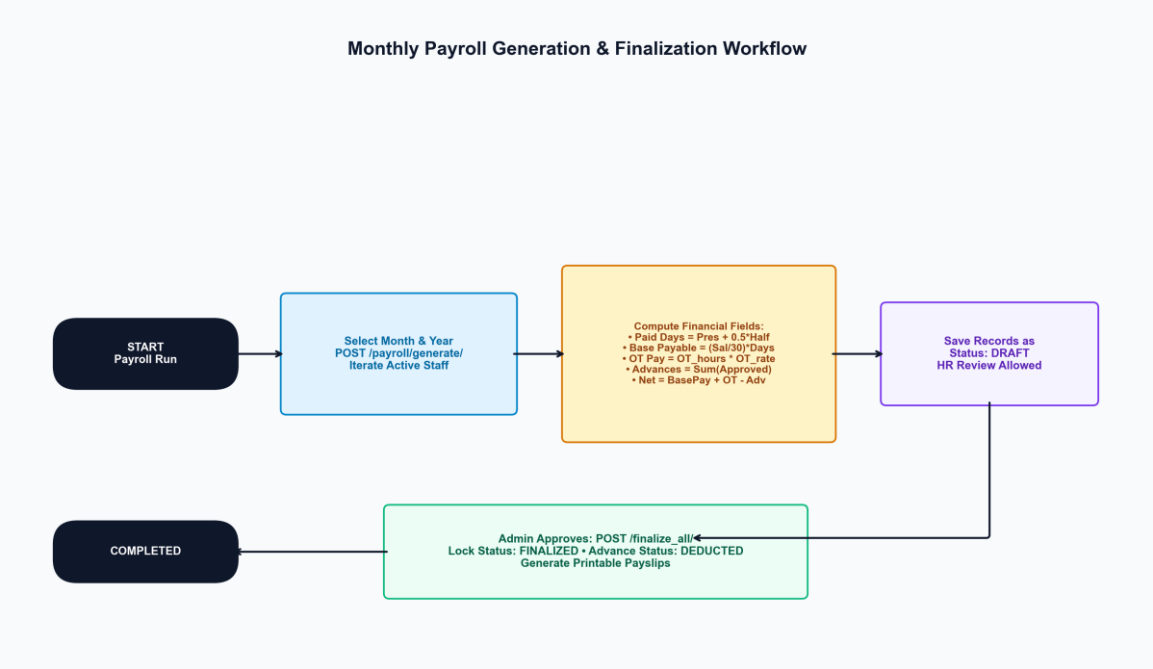


Figure 4.5: Monthly Payroll Computation and Locking Workflow

PART 5 — DATABASE DESIGN

5.1 Database Overview

The SanitaryApp database is architected on relational principles to enforce strict ACID (Atomicity, Consistency, Isolation, Durability) guarantees across industrial financial operations. In production, PostgreSQL 15+ serves as the enterprise database engine, while SQLite 3 is utilized for local development and rapid staging verification. All schema entities are managed via Django's Object-Relational Mapping (ORM) migrations with explicit database constraints, foreign key protections, and composite indexes.

5.2 Database Tables DDL & Schema Definitions

The following tables detail the exact column definitions, data types, constraints, and business roles for all seven entities in the database.

Table 5.1: accounts_user (System Operator Accounts)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique operator identifier.
username	VARCHAR(150)	NOT NULL, UNIQUE	Operator login handle (e.g. admin, supervisor_kiln).
password	VARCHAR(128)	NOT NULL	PBKDF2/SHA-256 salted and hashed cryptographic secret.
first_name	VARCHAR(150)	DEFAULT ''	Operator given name.
last_name	VARCHAR(150)	DEFAULT ''	Operator surname.
email	VARCHAR(254)	DEFAULT ''	Operator contact email address.
role	VARCHAR(20)	CHECK IN ('ADMIN','MANAGER','SUPERVISOR')	System role determining RBAC permissions matrix.
phone	VARCHAR(20)	DEFAULT ''	Contact phone number.
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active flag allowing immediate revocation.
date_joined	TIMESTAMPTZ	NOT NULL, CURRENT_TIMESTAMP	Timestamp of account provisioning.

Table 5.2: departments_department (Factory Production Divisions)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique department ID.
name	VARCHAR(100)	NOT NULL, UNIQUE	Unique factory division name (e.g. Moulding, Glazing, Kiln).
description	TEXT	DEFAULT ''	Operational scope and functional description of the department.
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Status toggle; inactive divisions hidden from rosters.
created_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Creation audit timestamp.
updated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW	Last modification audit timestamp.

Table 5.3: shifts_shift (Work Schedule Windows)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique shift ID.
name	VARCHAR(50)	NOT NULL, UNIQUE	Shift designation name (e.g. Morning, Evening, Night).
start_time	TIME	NOT NULL	Standard scheduled shift start time (e.g. 08:00:00).
end_time	TIME	NOT NULL	Standard scheduled shift conclusion time (e.g. 16:00:00).
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Shift active flag for roster filtering.
created_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Creation audit timestamp.
updated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW	Last modification audit timestamp.

Table 5.4: staff_staff (Factory Worker Master Directory)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique worker record ID.
employee_code	VARCHAR(30)	NOT NULL, UNIQUE, INDEX	Unique factory badge identifier (e.g. STF001).
full_name	VARCHAR(150)	NOT NULL	Legal full name of the factory worker.
phone	VARCHAR(20)	NOT NULL	Worker mobile telephone number.
joining_date	DATE	NOT NULL	Official date of factory employment commencement.
department_id	BIGINT	NOT NULL, FK -> departments (PROTECT)	Foreign key to assigned production department.
designation	VARCHAR(100)	NOT NULL	Job title (e.g. Kiln Operator, Glaze Sprayer, Packer).
shift_id	BIGINT	NULL, FK -> shifts (PROTECT)	Foreign key to assigned standard working shift.
salary_type	VARCHAR(20)	CHECK (salary_type = 'MONTHLY')	Compensation model (enforces standard monthly pay).
monthly_salary	NUMERIC(12,2)	NOT NULL, CHECK (monthly_salary >= 0)	Agreed base monthly compensation in currency units.
overtime_rate	NUMERIC(10,2)	NOT NULL, CHECK (overtime_rate >= 0)	Contractual overtime pay rate per hour.
status	VARCHAR(20)	CHECK IN ('ACTIVE','INACTIVE'), INDEX	Employment lifecycle status; inactive excluded from rosters.
created_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Worker record creation timestamp.
updated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW	Worker record last update timestamp.

Table 5.5: attendance_attendance (Daily Attendance & Overtime Ledger)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique attendance entry ID.
staff_id	BIGINT	NOT NULL, FK -> staff (PROTECT)	Foreign key referencing worker.
date	DATE	NOT NULL, INDEX	Calendar date of attendance record.
status	VARCHAR(20)	CHECK IN ('PRESENT','HALF_DAY','ABSENT','LEAVE')	Presence status (Present=1.0, Half-Day=0.5, Absent/Leave=0.0).
overtime_hours	NUMERIC(4,2)	CHECK (0.00 <= overtime_hours <= 12.00)	Incremental overtime worked beyond standard shift.
remarks	VARCHAR(255)	DEFAULT ''	Supervisor notes (e.g. Approved medical leave, Kiln extra).
created_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Attendance capture timestamp.
updated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW	Last modification timestamp.
CONSTRAINT	UNIQUE	UNIQUE (staff_id, date)	Composite unique constraint preventing duplicate marks.

Table 5.6: advances_advancesalary (Cash Advance & Loan Ledger)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique advance entry ID.
staff_id	BIGINT	NOT NULL, FK -> staff (PROTECT)	Foreign key referencing borrower worker.
amount	NUMERIC(12,2)	NOT NULL, CHECK (amount > 0)	Principal cash advance amount disbursed.
date	DATE	NOT NULL, INDEX	Calendar date cash advance was disbursed.

reason	VARCHAR(255)	DEFAULT ''	Justification stated by worker (e.g. Medical, Festival, Travel).
status	VARCHAR(20)	CHECK IN ('APPROVED','DEDUCTED',' CANCELLED')	Lifecycle state (Approved -> Deducted in Payroll / Cancelled).
created_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Disbursement entry timestamp.
updated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW	Last status change timestamp.

Table 5.7: payroll_payroll (Monthly Payroll Ledger & Snapshot)

Column Name	Data Type	Constraint	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing unique payroll record ID.
staff_id	BIGINT	NOT NULL, FK -> staff (PROTECT)	Foreign key referencing paid employee.
month	SMALLINT	NOT NULL, CHECK (1 <= month <= 12)	Calendar month of wage compensation.
year	SMALLINT	NOT NULL	Calendar year of wage compensation.
base_salary	NUMERIC(12,2)	NOT NULL, CHECK (base_salary >= 0)	Snapshot of worker's monthly salary at generation time.
payroll_days	NUMERIC(4,2)	NOT NULL, DEFAULT 30.00	Standard statutory base days per industrial month.
paid_days	NUMERIC(4,2)	CHECK (0 <= paid_days <= 31)	Aggregated earned days (Present*1.0 + HalfDay*0.5).
attendance_deduction	NUMERIC(12,2)	NOT NULL, DEFAULT 0.00	Financial deduction for unpaid/absent calendar days.
overtime_hours	NUMERIC(6,2)	NOT NULL, DEFAULT 0.00	Total overtime hours accumulated during the month.
overtime_rate	NUMERIC(10,2)	NOT NULL, DEFAULT 0.00	Snapshot of contractual hourly overtime rate.

overtime_amount	NUMERIC(12,2)	NOT NULL, DEFAULT 0.00	Total overtime compensation earned (hours * rate).
gross_salary	NUMERIC(12,2)	NOT NULL	Gross compensation (Base Salary + Overtime Amount).
advance_deduction	NUMERIC(12,2)	NOT NULL, DEFAULT 0.00	Total mid-month cash advances subtracted.
other_deductions	NUMERIC(12,2)	NOT NULL, DEFAULT 0.00	Miscellaneous deductions (uniform, damages, penalties).
net_salary	NUMERIC(12,2)	NOT NULL, CHECK (net_salary >= 0)	Final payable wage disbursed to employee.
status	VARCHAR(20)	CHECK IN ('DRAFT','FINALIZED')	Lifecycle flag (Draft = Editable; Finalized = Locked).
generated_at	TIMESTAMPTZ	NOT NULL, AUTO_NOW_ADD	Initial batch computation timestamp.
finalized_at	TIMESTAMPTZ	NULL	Timestamp of administrative finalization and lock.
CONSTRAINT	UNIQUE	UNIQUE (staff_id, month, year)	Guarantees exactly one payroll record per worker per month.

5.3 Entity Relationship Diagram (ERD)

Figure 5.1 illustrates the complete relational entity-relationship structure, primary keys, foreign keys, cardinality mappings, and integrity constraints enforced by the database.

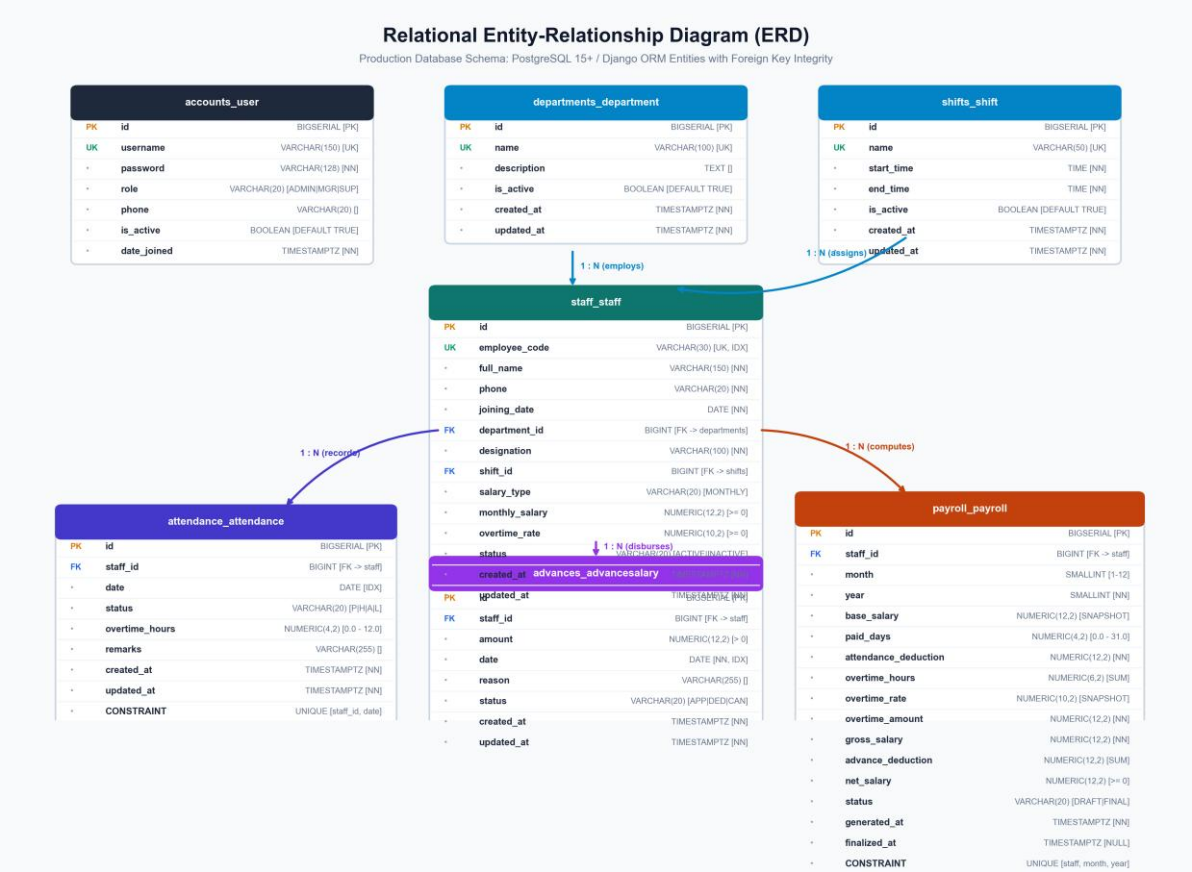


Figure 5.1: Relational Entity-Relationship Diagram (ERD) with Constraints

5.4 Data Relationships and Integrity Rules

The relational schema preserves financial consistency and operational safety through four tiers of integrity rules:

- **1. Cardinality:** Departments and Shifts relate 1-to-many with Staff; Staff relates 1-to-many with Attendance, AdvanceSalary, and Payroll.
- **2. Foreign Key Protection:** All foreign keys linking Staff to Attendance, Advances, and Payroll use on_delete=models.PROTECT. Attempting to delete a staff record with historical attendance or financial records is strictly blocked by the RDBMS.
- **3. Uniqueness Enforcement:** Unique constraints on (staff_id, date) prevent duplicate attendance logs; unique constraints on (staff_id, month, year) prevent multiple payroll entries.
- **4. Mathematical Invariants:** Check constraints enforce mathematical sanity at the RDBMS level: monthly_salary >= 0, overtime_hours BETWEEN 0 AND 12, amount > 0, and net_salary >= 0.

PART 6 — INTERFACE AND SCREEN DOCUMENTATION

This section documents the nine primary user interface screens implemented in the Flutter application, covering UI structure, user interactions, data presentations, and validation rules.

Screen 6.1: User Authentication Screen (LoginScreen)

Implementation Source File: lib/features/auth/views/login_screen.dart

Main UI Components: Factory ERP Logo, Username TextFormField with prefix icon, Password TextFormField with toggleable obscureText visibility icon, Remember Me checkbox, Sign In ElevatedButton with progress spinner.

User Actions & Triggers: Input credentials, toggle password visibility, submit form, trigger JWT authentication request to /api/login/.

Data Displayed: Status messages, authentication errors, current user session state.

Navigation Flow: On successful authentication, navigates to MainNavigationShell; on token failure, remains on LoginScreen.

Validation & Error States: Client-side validation requires minimum 3-character username and non-empty password; displays contextual error banner on 401 Unauthorized.

Screen 6.2: Executive Factory Dashboard (DashboardScreen)

Implementation Source File: lib/features/dashboard/views/dashboard_screen.dart

Main UI Components: Welcome header card displaying operator role badge, Today's Live Attendance Card with linear progress indicators, Quick Action Shortcuts Grid, Financial Snapshot Card.

User Actions & Triggers: Pull-to-refresh metrics, tap Quick Action shortcuts (Mark Today's Attendance, Register Staff, Give Advance, Run Payroll), tap tab switches.

Data Displayed: Active/Inactive worker counts, today's Present, Half-Day, Absent, Leave, Unmarked tallies, total OT hours, projected month-end net payroll expense, total unsettled advances.

Navigation Flow: Quick action buttons route directly to DailyRosterScreen, StaffFormScreen, AddAdvanceModal, or PayrollListScreen.

Validation & Error States: Handles network error states via ErrorView with retry button; displays shimmer loading states while fetching from /api/dashboard/kpi/.

Screen 6.3: Staff Master Directory (StaffListScreen & StaffDetailScreen)

Implementation Source File: lib/features/staff/views/staff_list_screen.dart, staff_detail_screen.dart

Main UI Components: Search bar with debounce, Department Filter chips, FloatingActionButton for adding staff, ListView of staff cards with employee badge, department chip, wage rate, status badge.

User Actions & Triggers: Type search queries (by code or name), tap department chip to filter, tap worker card to view full profile, tap toggle button to Activate/Deactivate worker.

Data Displayed: Worker code, full name, phone number, department name, shift name, monthly salary (masked for Supervisors), hourly OT rate, active status.

Navigation Flow: Tapping add FAB navigates to StaffFormScreen; tapping a worker navigates to StaffDetailScreen.

Validation & Error States: Confirmation dialog required prior to deactivating active employees; salary fields dynamically masked if user role is SUPERVISOR.

Screen 6.4: Worker Onboarding & Edit Form (StaffFormScreen)

Implementation Source File: lib/features/staff/views/staff_form_screen.dart

Main UI Components: Form container with TextFormFields: Employee Code, Full Name, Phone, Joining Date Picker, Department Dropdown, Shift Dropdown, Monthly Salary input, Overtime Rate input, Save button.

User Actions & Triggers: Fill worker attributes, select department and shift, select joining date via calendar picker dialog, tap Save to submit POST or PUT request to /api/staff/.

Data Displayed: Department list, shift list, current worker attributes if in edit mode.

Navigation Flow: Upon successful save, pops screen and refreshes StaffListScreen provider.

Validation & Error States: Employee code cannot be empty; monthly salary and OT rate must parse to valid numbers >= 0.00; phone must be valid digits; department must be selected.

Screen 6.5: Daily Attendance Roster & Bulk Marking (DailyRosterScreen)

Implementation Source File: lib/features/attendance/views/daily_roster_screen.dart

Main UI Components: Calendar Date Selector bar with next/prev day buttons, Department filter tabs, Bulk Quick-Fill buttons ('Mark All Present'), Staff Roster Card list with P/H/A/L SegmentedButtons, Overtime numerical stepper/field, Remarks input, Bottom floating Save Roster button.

User Actions & Triggers: Select calendar date, filter by department, tap status segment per worker, input overtime hours (0 to 12), tap 'Mark All Present', tap Save Roster to trigger atomic POST to /api/attendance/bulk_mark/.

Data Displayed: Active worker roster for chosen date, marked vs. unmarked status, existing overtime hours, computed daily presence summary.

Navigation Flow: Tab navigation to AttendanceHistoryScreen for monthly logs.

Validation & Error States: Overtime field restricts input to 0.00–12.00 hours; atomic transaction ensures all rows commit or none; unsaved changes warning on date change.

Screen 6.6: Historical Attendance Inquiry (AttendanceHistoryScreen)

Implementation Source File: lib/features/attendance/views/attendance_history_screen.dart

Main UI Components: Month/Year picker header, Worker dropdown filter, Department filter, Attendance calendar/history ListView, Status colored badges (Green: Present, Amber: Half-Day, Red: Absent, Blue: Leave).

User Actions & Triggers: Select month and year, filter by specific worker, view total days present, half-days, and cumulative overtime hours.

Data Displayed: Historical attendance records from /api/attendance/, status breakdowns, overtime aggregations.

Navigation Flow: Invoked from MainNavigationShell drawer or roster screen header.

Validation & Error States: Validates month (1-12) and year bounds; displays empty state graphic if no logs exist for selected period.

Screen 6.7: Salary Advances Ledger & Disbursal Modal (AdvancesListScreen & AddAdvanceModal)

Implementation Source File: lib/features/advances/views/advances_list_screen.dart, add_advance_modal.dart

Main UI Components: Summary metrics card (Total Disbursed, Approved, Deducted), Staff search and month filter, Advances Card list, Add Advance Modal with Worker dropdown, Amount field, Date picker, Reason input.

User Actions & Triggers: Tap Add Advance FAB, select worker, enter cash advance amount, select date, submit POST to /api/advances/, cancel advance via delete action.

Data Displayed: Advance ID, worker code and name, amount in currency units, disbursal date, stated reason, status badge (APPROVED, DEDUCTED, CANCELLED).

Navigation Flow: Modal bottom sheet overlays AdvancesListScreen.

Validation & Error States: Amount must be strictly greater than 0.00; worker must be selected; cancellation prohibited once payroll is FINALIZED.

Screen 6.8: Monthly Payroll Processing & Payslip Viewer (PayrollListScreen & PayslipDetailModal)

Implementation Source File: lib/features/payroll/views/payroll_list_screen.dart, payslip_detail_modal.dart

Main UI Components: Month/Year selector header, Aggregated Financial Summary card (Net Payroll, Gross, Advances, OT Pay), 'Generate Payroll' button, 'Finalize All' button, Payroll card list, Detailed Payslip Bottom Sheet.

User Actions & Triggers: Select target month and year, tap 'Generate Payroll' to trigger batch calculation, review draft payslips, tap individual payslip card to inspect calculation breakdown, tap 'Finalize' or 'Finalize All' to lock payroll.

Data Displayed: Base salary, paid days, attendance deduction, overtime hours, overtime pay, gross salary, advance deduction, other deductions, net payable salary, status (DRAFT / FINALIZED).

Navigation Flow: Payslip modal displays itemized salary breakdown; print/share payslip action.

Validation & Error States: Finalization restricted exclusively to ADMIN role; finalized payroll records cannot be recomputed or edited; Net salary cannot be negative.

Screen 6.9: Factory Master Settings (Departments, Shifts, User Management)

Implementation Source File: lib/features/departments/views/departments_screen.dart, shifts/views/shifts_screen.dart, auth/views/users_screen.dart

Main UI Components: Tabbed configuration interface: Department management list with add/edit dialog, Shift schedule list with time-pickers, User Administration table with role assignment (Admin, Manager, Supervisor).

User Actions & Triggers: Add new factory department, configure shift start/end times, provision new system operators, assign operational roles, deactivate accounts.

Data Displayed: Department lists, shift time windows, user directory with role badges and active toggles.

Navigation Flow: Accessible via Navigation Drawer for authorized administrative operators.

Validation & Error States: Unique constraint enforcement on department and shift names; user administration strictly guarded by IsAdminRole permission class.

PART 7 — TESTING REQUIREMENTS

Table 7.1 establishes a comprehensive test case matrix covering functional operations, validation boundaries, concurrency conditions, and error recovery.

Test ID	Test Scenario	Test Input	Expected Result
TC-AUTH-01	Valid Operator Authentication	username: 'admin', password: 'valid_password'	HTTP 200 OK; access & refresh tokens issued; role verified as ADMIN.
TC-AUTH-02	Invalid Password Attempt	username: 'admin', password: 'wrong_password'	HTTP 401 Unauthorized; error banner displayed; no tokens issued.
TC-AUTH-03	Access Protected Endpoint with Expired Token	Expired Bearer token in header	Dio interceptor catches 401; automatically requests /api/token/refresh/.
TC-AUTH-04	Role Authorization Enforcement	Supervisor invokes POST /api/payroll/generate/	HTTP 403 Forbidden; view rejected by IsAdminRole permission class.
TC-STF-01	Register New Staff with Valid Details	Code: 'STF099', Dept: 1, Salary: 18000, OT: 90	HTTP 201 Created; staff member inserted; daily_salary computed as 600.00.
TC-STF-02	Duplicate Employee Code Rejection	Code: 'STF001' (already existing in DB)	HTTP 400 Bad Request; unique constraint error returned to client.
TC-STF-03	Negative Wage Validation	monthly_salary: -500.00, overtime_rate: 80.00	Client form validation flags field; backend CheckConstraint rejects save.
TC-ATT-01	Fetch Daily Roster for Date	date: '2026-10-14', department: null	HTTP 200 OK; returns array of active staff with marked status and OT.
TC-ATT-02	Atomic Bulk Attendance Marking	10 staff records with statuses (P, H, A)	HTTP 200 OK; transaction commits 10 rows; summary metrics updated.
TC-ATT-03	Overtime Boundary Check (Valid)	overtime_hours: 4.50	HTTP 200 OK; saved; overtime pay computed at month end.
TC-ATT-04	Overtime Boundary Check (Invalid Exceeded)	overtime_hours: 14.00 (limit is 12.00)	HTTP 400 Bad Request; validation error 'Overtime hours must be <= 12'.
TC-ATT-05	Duplicate Date Upsert Check	Submit attendance for STF001 on same date twice	HTTP 200 OK; second request updates existing row via update_or_create.
TC-ADV-01	Disburse Valid Cash Advance	staff: 1, amount: 3000.00, date: '2026-10-15'	HTTP 201 Created; AdvanceSalary saved with status 'APPROVED'.

TC-ADV-02	Negative Advance Amount Rejection	staff: 1, amount: -100.00	HTTP 400 Bad Request; check constraint advance_amount_positive triggers.
TC-PAY-01	30-Day Base Pay Calculation	Base: 30000, Present: 25, Half: 2 (Paid=26)	DailyWage=1000; BasePayable=26000; AttnDeduction=4000; verified exact.
TC-PAY-02	Overtime Pay Calculation	OT Hours: 20.0, OT Rate: 150.00	Overtime Amount computed as 3000.00 (20 * 150.00).
TC-PAY-03	Advance Deduction in Payroll	Approved advances in month: 5000.00	advance_deduction field populated with 5000.00; subtracted from net.
TC-PAY-04	Net Salary Floor at Zero	Gross: 10000, Total Advances: 12000	Net Salary computed as max(0.00, -2000) = 0.00; non-negative invariant met.
TC-PAY-05	Protect Finalized Payroll Record	POST /payroll/generate/ for finalized month	Calculator skips finalized records; preserves locked fiscal data.
TC-PAY-06	Advance Lifecycle Transition	Admin executes POST /payroll/finalize_all/	Payrolls marked FINALIZED; associated advances transition to DEDUCTED.
TC-SEC-01	SQL Injection Resistance	Query: '1 OR 1=1; DROP TABLE staff_staff;'	Handled securely via Django ORM parameterized queries; no SQL execution.
TC-SEC-02	Supervisor Masked Salary Check	Supervisor calls GET /api/staff/1/	Serializer masks monthly_salary and overtime_rate; returns '***'.
TC-NET-01	Offline Device Handling in Flutter	Device Wi-Fi disabled; attempt save	Dio throws DioException; UI displays offline alert; queues retry.
TC-NET-02	Backend 500 Error Recovery	Database connectivity temporarily lost	StandardJSONRenderer packages 500 into standard format; UI shows ErrorView.

PART 8 — SECURITY AND LIMITATIONS

8.1 Security Architecture & Authentication

The SanitaryApp platform incorporates defense-in-depth principles across all architectural tiers:

- **Password Hashing:** User credentials are protected using Django's PBKDF2 algorithm with SHA-256 hash signatures and unique cryptographic salts.
- **Cryptographic Token Standard:** Stateless token-based authentication using JSON Web Tokens (JWT) signed with HMAC-SHA256 (HS256). Access tokens expire after 24 hours, while refresh tokens expire after 7 days.
- **Granular RBAC Authorization:** Every REST endpoint is protected by custom DRF permission classes (IsAdminRole, IsManagerOrAdmin, IsSupervisorOrAbove) validating user claims on every request.
- **CORS Hardening:** Cross-Origin Resource Sharing (CORS) is configured to permit communication exclusively from authorized enterprise domains and client hosts.

8.2 Financial Integrity & Concurrency Controls

To safeguard industrial financial operations against corruption or race conditions:

- **Atomic Database Transactions:** Bulk attendance marking and payroll finalization execute inside database transactions (@transaction.atomic). Any unexpected error rolls back the entire batch.
- **Referential Protection:** Foreign keys in staff_staff, attendance_attendance, advances_advancesalary, and payroll_payroll enforce on_delete=models.PROTECT, preventing accidental deletion of historical wage data.
- **Composite Unique Indexes:** Composite unique constraints (staff_id, date) and (staff_id, month, year) prevent duplicate attendance logs and duplicate payroll items.
- **Wage Data Masking:** Supervisors are prohibited from viewing worker wages; financial attributes are masked at the API serializer tier.

8.3 Known Technical Limitations and Gaps

To maintain technical honesty, the following gaps in the current implementation baseline are formally documented:

- **Absence of Physical Biometric Device Driver:** The current system relies on supervisor manual entry via tablet roster; it does not yet interface directly with USB or TCP/IP biometric optical fingerprint or facial recognition hardware scanners.
- **Manual Banking Disbursement:** While payroll is calculated and locked, actual wage disbursement is executed via manual cash or bank NEFT wire transfers; no direct banking Open API gateway is yet connected.
- **Single Currency Assumption:** All monetary fields assume Indian Rupee (INR) denomination; multi-currency support is not yet implemented.
- **Single Factory Tenant Scope:** The current database design serves a single factory unit; multi-tenant enterprise clustering across geographically distributed factories is reserved for future versions.

PART 9 — CONCLUSION AND FUTURE SCOPE

9.1 Project Conclusion

The Sanitary & Ceramic Factory Workforce & Payroll Management System (SanitaryApp) provides a complete, modern, and mathematically verified digital solution to the operational challenges of industrial manufacturing workforce administration. By replacing vulnerable paper muster rolls and disconnected spreadsheets with a decoupled Flutter client and Django REST backend, the system achieves sub-second attendance logging, automated overtime tracking, zero-leakage advance loan reconciliation, and formula-driven 30-day payroll calculations.

The architecture enforces strict Role-Based Access Control, audit logging, and relational ACID integrity, ensuring that factory owners and plant managers retain complete transparency over operational labor costs while eliminating wage disputes among factory workers.

9.2 Future Scope & Planned Enhancements

The roadmap for subsequent engineering versions comprises the following key extensions:

- **1. Direct Biometric IoT Hardware Integration:** Development of an IoT middleware service connecting physical biometric turnstiles (ZKTeco / eSSL devices) via TCP/IP sockets to push real-time punch timestamps into the attendance roster.
- **2. Automated Banking API Payout Gateway:** Direct integration with corporate banking APIs (e.g. RazorpayX, ICICI Corporate Banking, Cashfree) to execute automated 1-click batch electronic wage disbursements upon payroll finalization.
- **3. Worker Mobile Self-Service Portal:** A dedicated, lightweight Flutter Progressive Web App (PWA) / mobile APK for factory workers to view their monthly attendance summaries, advance balances, and download PDF payslips directly on their personal mobile devices.
- **4. Predictive Workforce Analytics:** Implementation of machine learning models analyzing historical production schedules, seasonal heat factors, and absenteeism trends to forecast labor requirements and optimize shift scheduling.